

Análisis de Algoritmos I

Estrategias de diseño de algoritmos

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Estrategias de diseño de algoritmos

- El diseño de algoritmos implica la **creación de un procedimiento** o fórmula paso a paso para resolver un problema.
- El objetivo es **desarrollar un algoritmo** que sea **correcto** (que produzca el resultado correcto) y **eficiente** (que utilice recursos mínimos).
- El proceso de diseño incluye **analizar el problema, elegir el enfoque adecuado** y refinar iterativamente el algoritmo para mejorar su rendimiento.

Estrategias de diseño de algoritmos

También existen distintos **tipos de problemas** a resolver:

- Búsqueda
- Ordenamiento
- Problemas de grafos
- Optimización
- Procesamiento de cadenas de texto
- ...

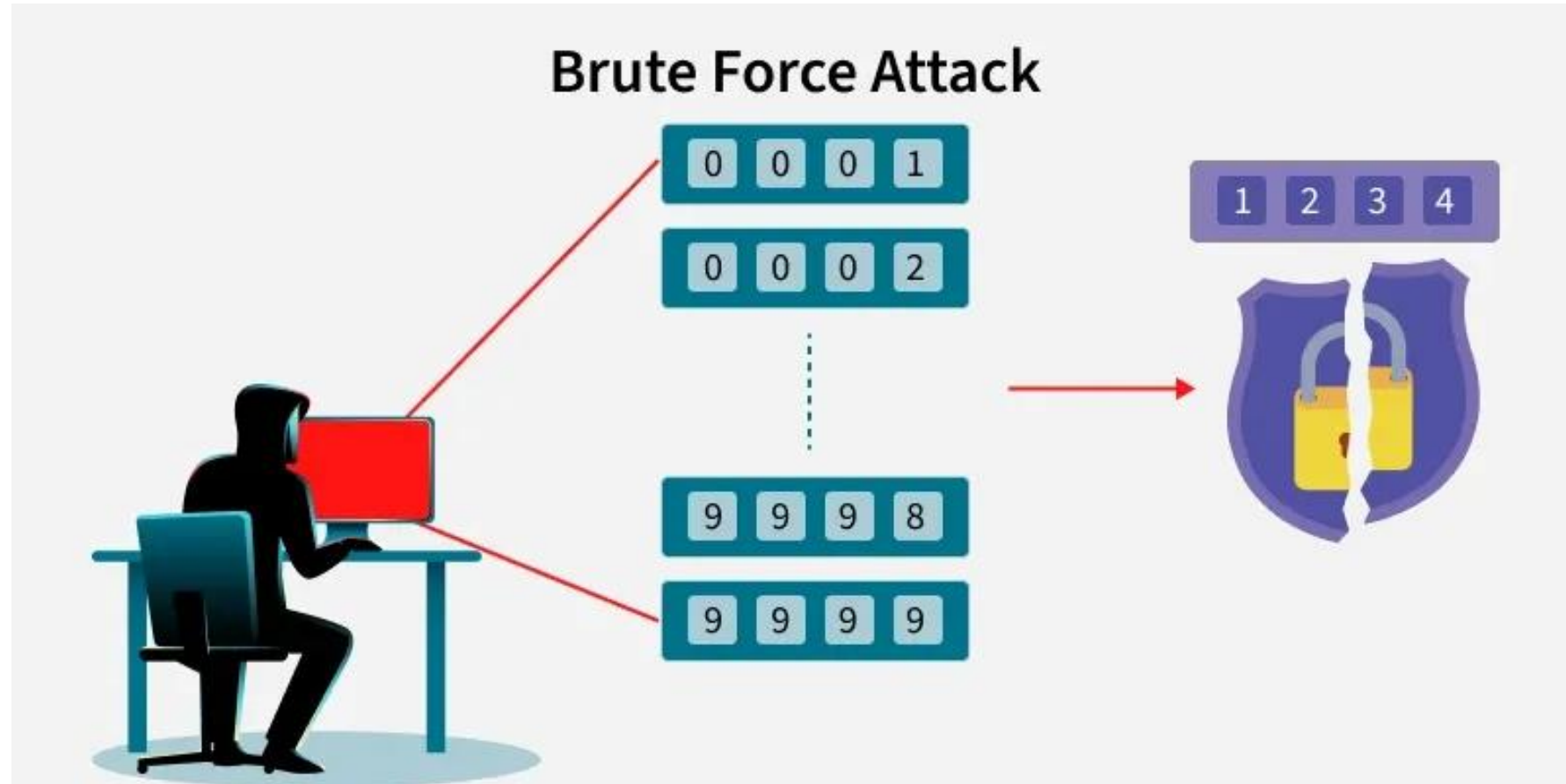
Para cada uno de ellos se pueden implementar distintas estrategias.

Estrategias de diseño de algoritmos

Algunas estrategias de diseño
de algoritmos:

1. Fuerza bruta
2. Divide y vencerás
3. Algoritmos voraces (Greedy)
4. Programación dinámica
5. *Entre otras...*

1. Fuerza bruta



1. Fuerza bruta

- Consiste en comprobar sistemáticamente **todas las posibles soluciones** para comprobar si cada una de ellos satisface o no el planteamiento del problema.
- Es **sencilla de implementar** y siempre encontrará una solución si existe, pero **los costos de implementación serán proporcionales al número de posibles soluciones**, que en muchos problemas prácticos tiende a crecer rápidamente a medida que aumenta el tamaño del problema.
- Suele implementarse usando **algoritmos iterativos con bucles** sencillos o anidados.

1. Fuerza bruta

Para muchos problemas del mundo real, el número de posibles soluciones es inmensamente grande:

- Si buscamos los **divisores de un número n de 16 dígitos**, se requerirían al menos 10^{15} instrucciones, lo que tardaría **varios días** en una PC común. Si el número es de 64 bits (aprox. **19 dígitos**), ¡la búsqueda tardaría **10 años!**.
- Si queremos saber cuántas veces se pueden **reordenar 10 letras**, hay $10!$ (3.6 millones) de candidatos, algo que una PC resuelve en **menos de un segundo**. Pero si aumentamos a **20 letras**, el número de combinaciones ahora es de $20!$ (2.4 quintillones), lo que elevaría el tiempo de espera nuevamente a **10 años**.

1. Fuerza bruta

A estos fenómenos se le conoce comúnmente como:

- **Explosión Combinatoria:** El fenómeno donde un pequeño incremento en el tamaño de la entrada n provoca un crecimiento masivo y "explosivo" en el número de soluciones posibles a probar.
- **La Maldición de la Dimensionalidad:** Se refiere a cómo la complejidad del problema se vuelve inmanejable a medida que añadimos más variables o dimensiones a los datos.

1. Fuerza bruta

- La fuerza bruta solo es útil cuando el "**espacio de búsqueda**" es **pequeño**.
- O cuando la **simplicidad** de implementación es más importante que la velocidad de procesamiento.
- Para todo lo demás, necesitamos estrategias más ***inteligentes***.

1. Fuerza bruta: Ejemplos

Algoritmo	Técnica de Implementación	Estrategia de Diseño
✓ Búsqueda Lineal	Iterativa (bucles)	Fuerza Bruta
✓ Selection Sort	Iterativa (bucles anidados)	Fuerza Bruta
✓ Insertion Sort	Iterativa (bucles anidados)	Fuerza Bruta / Decrementa y Vencerás / Incremental
Bubble Sort	Iterativa (bucles anidados)	Fuerza Bruta
Cocktail Sort	Iterativa (bucles anidados)	Fuerza Bruta (Variante de Burbuja)
Shell Sort	Iterativa (bucles múltiples)	Fuerza bruta / Decrementa y Vencerás (Mejora de Insertion sort)

2. Divide y vencerás



2. Divide y vencerás

- Es uno de los paradigmas más importantes de diseño algorítmico.
- **Divide** un problema en dos subproblemas más pequeños de modo que cada subproblema sea idéntico al problema original, excepto porque su tamaño de entrada es menor.
- Luego, ambos subproblemas se **resuelven**.
- Al final, las soluciones a cada uno de los subproblemas se **combinan** para dar una solución al problema original.

2. Divide y vencerás

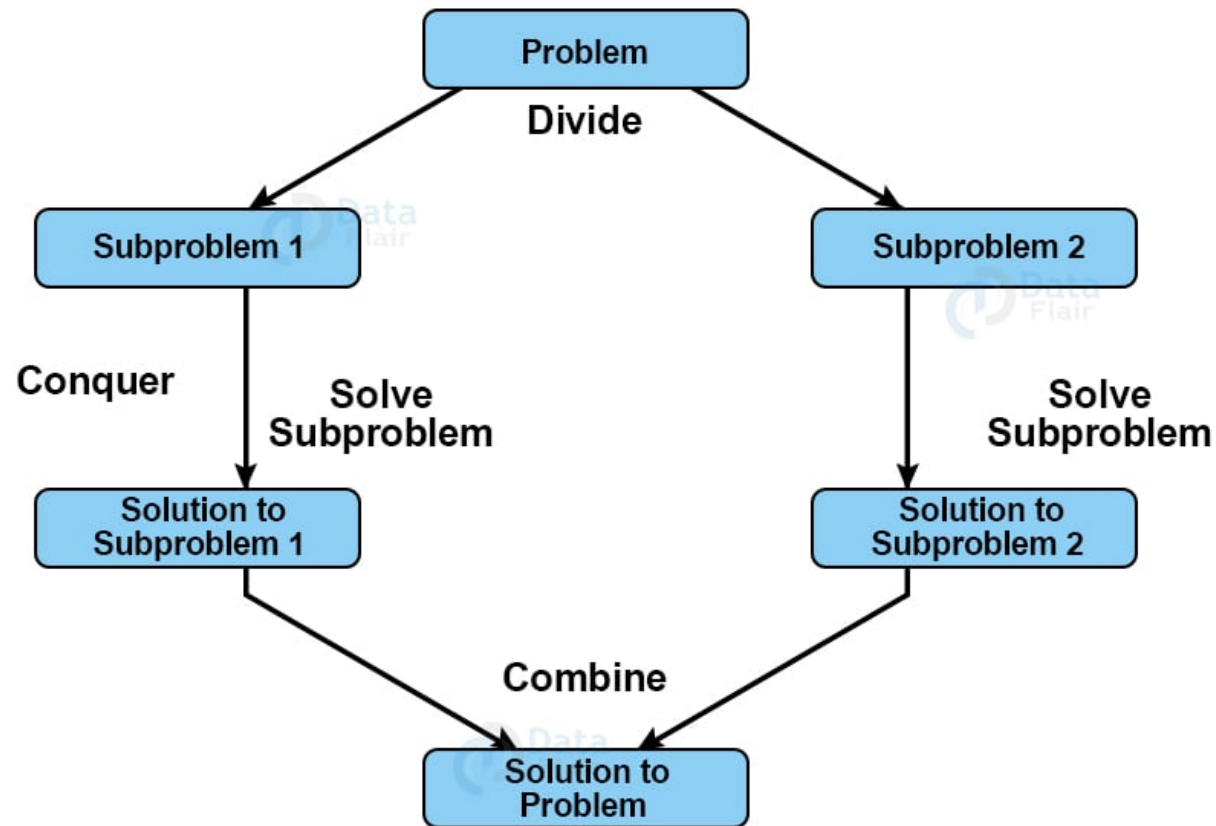
- Una cuestión muy importante sobre esta estrategia es que **divide el problema original en dos subproblemas** idénticos al problema original. Por lo tanto, también estos dos subproblemas pueden resolverse aplicando la estrategia divide-y-vencerás.
- Para decirlo con otras palabras, se resuelven de manera **recurrente** o **recursiva**.
- **Ventajas:** Puede resolver diferentes problemas en menos tiempo y por lo tanto, con menor complejidad.
- **Desventajas:**
 - A veces puede ser más complicado que un enfoque iterativo básico
 - Las llamadas recursivas utilizan una *pila*, lo que significa más complejidad espacial.
 - A veces se requieren hacer más cálculos.

2. Divide y vencerás

En general, un algoritmo divide-y-vencerás consta de tres pasos:

- **Paso 1 - Dividir.** Si el tamaño del problema es pequeño, éste se resuelve aplicando algún método directo; en caso contrario, el problema original **se divide** en dos subproblemas, de preferencia del mismo tamaño.
- **Paso 2 – Vencer/Conquistar.** Los dos subproblemas **se resuelven** de manera recurrente aplicando el algoritmo divide-y-vencerás a los subproblemas. Al final estos subproblemas son tan pequeños que cada uno puede resolverse directa y fácilmente. A esto se le llama **caso base** (base case).
- **Paso 3 - Combinar.** Las soluciones de los subproblemas **se fusionan** en una solución del problema original.

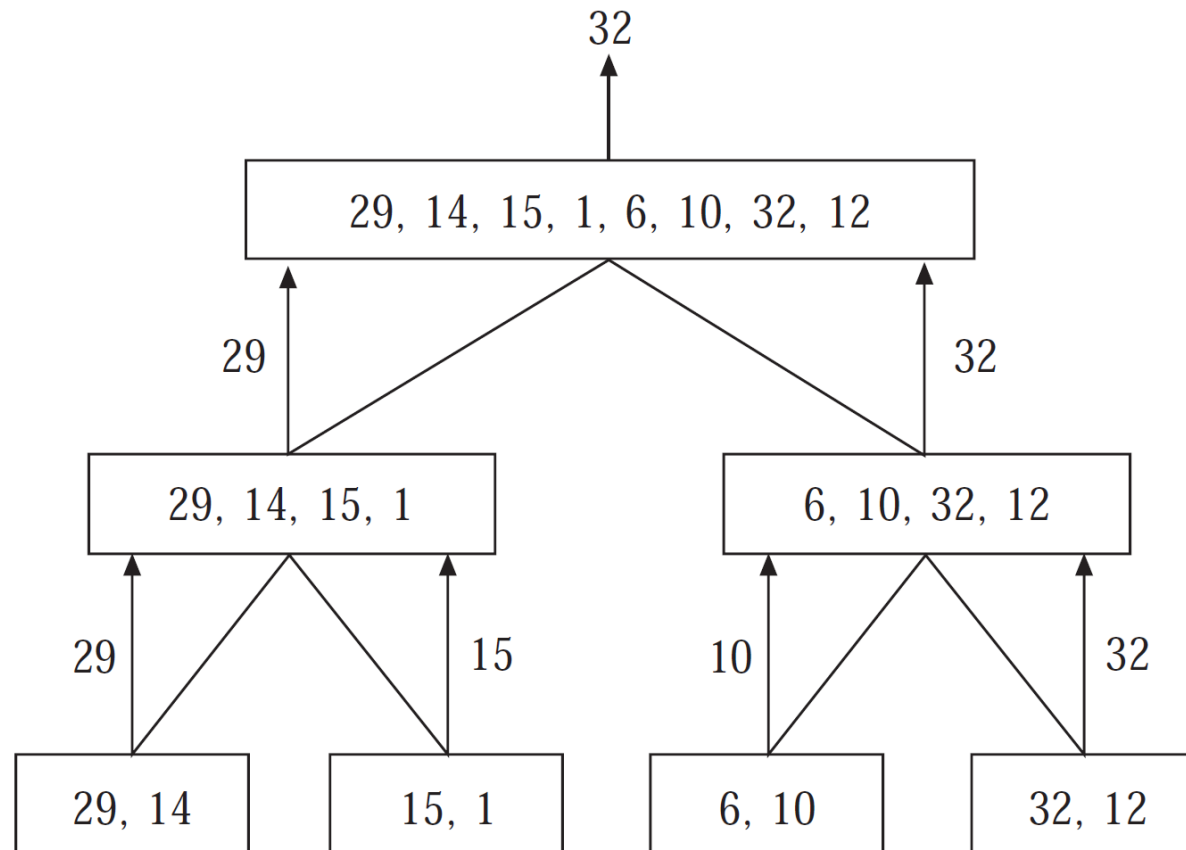
2. Divide y vencerás



Divide and Conquer Approach

2. Divide y vencerás

FIGURA 4-1 Estrategia divide-y-vencerás para encontrar el máximo de ocho números.



2. Divide y vencerás: Ejemplos

Algoritmo	Técnica de Implementación	Estrategia de Diseño
✓ Búsqueda binaria	Iterativa (bucles)	Divide y vencerás
Búsqueda binaria	Recursión	Divide y vencerás
Merge sort	Recursión	Divide y vencerás
Quick sort	Recursión	Divide y vencerás